well as the list of patterns that corresponds to the architectural solution. If an appropriate pattern is found, adapt the design solution proposed and build a design model element that adequately represents it. Be certain to apply step 7.

5. **Repeat steps 2 to 4 until all broad problems have been addressed.** The implication is to begin with the big picture and elaborate to solve problems at increasingly more detailed levels.
6. **If user interface design problems have been isolated (this is almost always the case), search the many user interface design pattern repositories for appropriate patterns.** Proceed in a manner similar to steps 3 to 5.
7. **Regardless of its level of abstraction, if a pattern language and/or patterns repository or individual pattern shows promise, compare the problem to be solved against the existing pattern(s) presented.** Be certain to examine context and forces to ensure that the pattern does, in fact, provide a solution that is amenable to the problem.
8. **Be certain to refine the design as it is derived from patterns using design quality criteria as a guide.**

Although this design approach is fundamentally top-down, Gillis [Gil06] suggests that "it's more organic than that, more inductive than deductive, more bottom-up than top-down." In addition, the pattern-based approach must be used in conjunction with other software design concepts and techniques.

14.2.4 Building a Pattern-Organizing Table

As pattern-based design proceeds, you may encounter trouble organizing and categorizing candidate patterns from multiple pattern languages and repositories. To help organize your evaluation of candidate patterns, Microsoft [Mic13b] suggests the creation of a *pattern-organizing table* that takes the general form shown in Figure 14.2.

A pattern-organizing table can be implemented as a spreadsheet model using the form shown in the figure. An abbreviated list of problem statements, organized by data and content, architecture, component level, and user interface issues, is presented in the left-hand (shaded) column. Four pattern types—database, application, implementation, and infrastructure—are listed across the top row. The names of candidate patterns are noted in the cells of the table.

To provide entries for the organizing table, you'll search through pattern languages and repositories for patterns that address an individual problem statement. When one or more candidate patterns is found, it is entered in the row corresponding to the problem statement and the column that corresponds to the pattern type. The name of the pattern is entered as a hyperlink to the URL of the Web address that contains a complete description of the pattern.

14.2.5 Common Design Mistakes

Several common mistakes may occur when pattern-based design is used. In some cases, not enough time has been spent to understand the underlying problem and its context and forces, and you may select a pattern that looks right but is inappropriate for the solution required. Once the wrong pattern is selected, you refuse to see your error and force-fit the pattern. In other cases, the problem has forces that are not

FIGURE 14.2

A pattern-organizing table
Source: Adapted from Microsoft, "Prescriptive Architecture: Integration and Patterns," MSDN, May 2004.

	Database	Application	Implementation	Infrastructure
Data/Content				
Problem statement ...	PatternName(s)		PatternName(s)	
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...	PatternName(s)			PatternName(s)
Architecture				
Problem statement ...		PatternName(s)		
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...				
Component-level				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...				PatternName(s)
Problem statement ...		PatternName(s)	PatternName(s)	
User interface				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	

considered by the pattern you've chosen, resulting in a poor or erroneous fit. Sometimes a pattern is applied too literally and the required adaptations for your problem space are not implemented.

Can these mistakes be avoided? In most cases, the answer is yes. Every good designer looks for a second opinion and welcomes review of her work. The review techniques discussed in Chapter 16 can help to ensure that the pattern-based design you've developed will result in a high-quality solution for the software problem to be solved.

14.3 ARCHITECTURAL PATTERNS

If a house builder decides to construct a center-hall colonial, there is a single architectural style that can be applied. The details of the style (e.g., number of fireplaces, façade of the house, placement of doors and windows) can vary considerably, but once the decision on the overall architecture of the house is made, the style is imposed on the design.[5]

Architectural patterns are a bit different. For example, every house (and every architectural style for houses) employs a **Kitchen** pattern. The **Kitchen** pattern and

5 This implies that there will be a central foyer and hallway, that rooms will be placed to the left and right of the foyer, that the house will have two (or more) stories, that the bedrooms of the house will be upstairs, and so on. These "rules" are imposed once the decision is made to use the center-hall colonial style.

patterns it collaborates with address problems associated with the storage and preparation of food, the tools required to accomplish these tasks, and rules for placement of these tools relative to workflow in the room. In addition, the pattern might address problems associated with countertops, lighting, wall switches, a central island, flooring, and so on. Obviously, there is more than a single design for a kitchen, often dictated by the context and system of forces. But every design can be conceived within the context of the "solution" suggested by the **Kitchen** pattern.

A software architecture may have several architectural patterns that address issues such as concurrency, persistence, and distribution. Before a representative architectural pattern can be chosen in a specific domain, it must be assessed for its appropriateness for the application and the overall architectural style, as well as the context and system of forces that it specifies.

14.4 COMPONENT-LEVEL DESIGN PATTERNS

Component-level design patterns provide you with proven solutions that address one or more subproblems extracted from the requirements model. In many cases, design patterns of this type focus on some functional element of a system. For example, the **SafeHomeAssured.com** application must address the following design subproblem: *How do I get product specifications and related information for any* SafeHome *device?*

Having stated the subproblem that must be solved, you should now consider context and the system of forces that affect the solution. Examining the appropriate requirements model use case, you learn that the consumer uses the specification for a *SafeHome* device (e.g., a security sensor or camera) for informational purposes. However, other information that is related to the specification (e.g., pricing) may be used when e-commerce functionality is selected.

The solution to the subproblem involves a search. Because searching is a very common problem, it should come as no surprise that there are many search-related patterns. Looking through several patterns repositories, you find the following patterns, along with the problem that each solves:

AdvancedSearch. Users must find a specific item in a large collection of items.

HelpWizard. Users need help on a certain topic related to the website or when they need to find a specific page within the site.

SearchArea. Users must find a page.

SearchTips. Users need to know how to control the search engine.

SearchResults. Users must process a list of search results.

SearchBox. Users must find an item or specific information.

For **SafeHomeAssured.com**, the number of products is not particularly large, and each has a relatively simple categorization, so **AdvancedSearch** and **HelpWizard** are probably not necessary. Similarly, the search is simple enough not to require **SearchTips**. The description of **SearchBox**, however, is given (in part) as:

Search Box

(*Adapted from* www.welie.com/patterns/showPattern.php?patternID=search)

Problem:	The users need to find an item or specific information.
Motivation:	Any situation in which a keyword search is applied across a collection of content objects organized as Web pages.
Context:	Rather than using navigation to acquire information or content, the user wants to do a direct search through content contained on multiple Web pages. Any website that already has primary navigation. User may want to search for an item in a category. User might want to further specify a query.
Forces:	The website already has primary navigation. Users may want to search for an item in a category. Users might want to further specify a query using simple Boolean operators.
Solution:	Offer search functionality consisting of a search label, a keyword field, a filter if applicable, and a "go" button. Pressing the return key has the same function as selecting the go button. Also provide Search Tips and examples in a separate page. A link to that page is placed next to the search functionality. The edit box for the search term is large enough to accommodate three typical user queries (typically around 20 characters). If the number of filters is more than 2, use a combo box for filters selection, otherwise a radio button. The search results are presented on a new page with a clear label containing at least "Search results" or similar. The search function is repeated in the top part of the page with the entered keywords, so that the users know what the keywords were.

The pattern description continues with other entries, as described in Section 14.1.3.

The pattern goes on to describe how the search results are accessed, presented, matched, and so on. Based on this, the **SafeHomeAssured.com** team can design the components required to implement the search or (more likely) acquire existing reusable components.

SAFEHOME



Applying Patterns

The scene: Informal discussion during the design of a software increment that implements sensor control via the Internet for **SafeHomeAssured.com.**

The players: Jamie, responsible for design, and Vinod, **SafeHomeAssured.com** chief system architect.

The conversation:

Vinod: So how is the design of the camera control interface coming along?

Jamie: Not too bad. I've designed most of the capability to connect to the actual sensors without too many problems. I've also started thinking about the interface for the users to move,

pan, and zoom the cameras from a remote device, but I'm not sure I've got it right yet.

Vinod: What have you come up with?

Jamie: Well, the requirements are that the camera control needs to be highly interactive—as the user moves the control, the camera should move as soon as possible. So, I was thinking of having a set of buttons laid out like a normal camera, but when the user clicks them, it controls the camera.

Vinod: Hmmm. Yeah, that would work, but I'm not sure it's right—for each click of a control you need to wait for the whole client-server communication to occur, and so you won't get a good sense of quick feedback.

Jamie: That's what I thought—and why I wasn't very happy with the approach, but I'm not sure how else I might do it.

Vinod: Well, why not just use the **Interactive-DeviceControl** pattern?

Jamie: Uhmmm—what's that? I haven't heard of it.

Vinod: It's basically a pattern for exactly the problem you are describing. The solution it proposes is basically to create a control connection to the server with the device, through which control commands can be sent. That way you don't need to send normal HTTP requests. And the pattern even shows how you can implement this using some simple AJAX techniques. You have some simple client-side JavaScript that communicates directly with the server and sends the commands as soon as the user does anything.

Jamie: Cool! That's just what I needed to solve this thing. Where do I find it?

Vinod: It's available in an online repository. Here's the URL.

Jamie: I'll go check it out.

Vinod: Yep—but remember to check the consequences field for the pattern. I seem to remember that there was something in there about needing to be careful about issues of security. I think it might be because you are creating a separate control channel and so bypassing the normal Web security mechanisms.

Jamie: Good point. I probably wouldn't have thought of that! Thanks.

14.5 Anti-Patterns

Design patterns provide you with proven solutions that address one or more problems extracted from the requirements model. *Anti-patterns* describe commonly used solutions to design problems that usually have negative effects on software quality. In other words, they describe bad solutions to the design problems or at the very least describe the consequence of applying a design pattern in the wrong context. Anti-patterns can provide tools to help developers recognize when these problems exist and may provide detailed plans for reversing the underlying problem causes and implementing better solutions to these problems [Bro98]. Anti-patterns can provide valuable guidance to developers when they are looking for ways to refactor software products to improve their quality. In addition, they can be used by technical reviewers (Chapter 16) to uncover area of concern.

Brown and his colleagues [Bro98] make the following comparisons between pattern and anti-pattern descriptions. Design patterns are usually written from the bottom up. A design pattern description starts with a recurring solution to a problem and then adds forces, symptoms, and context elements of the situation in which the solution is to be applied. Anti-patterns are written from the top down.

An anti-pattern description takes a recurring design problem, or a bad development practice, and then lists its symptoms and negative consequences. It is then possible to include a recommended procedure for reducing the consequences documented in the anti-pattern. **The Blob** anti-pattern is presented in the following example.

The Blob

(*Adapted from* http://antipatterns.com/briefing/sld024.htm)

Problem:

Single class with large number of attributes, operations, or both.

Symptoms and Consequences:

- A disparate collection of unrelated attributes and operations encapsulated in a single class.
- An overall lack of cohesiveness of the attributes and operations is typical of **The Blob**.
- A single controller class with associated simple, data-object classes.
- A single controller class encapsulates the entire functionality like a procedural main program.
- **The Blob** limits the ability to modify the system without affecting the functionality of other objects.
- Modifications to other objects in the system are also likely to impact **The Blob's** class.
- **The Blob** class is typically too complex for reuse and testing.
- **The Blob** class may be expensive to load into memory and uses excessive resources.

Typical Causes:

- Lack of an object-oriented architecture.
- The team may be lacking appropriate abstraction skills.
- Lack of defined software architecture.
- Lack of programming language support for architectural design.
- In iterative projects, developers tend to add little pieces of functionality to existing classes.
- Defining system architecture during requirements analysis often leads to **The Blob**.

Solution:

- The solution involves a form of re-factoring.
- The key is to move behavior away from **The Blob**.
- Reallocate behavior to other data objects in ways that make **The Blob** less complex.

A complete discussion of anti-patterns is beyond the scope of this book. Several anti-patterns with colorful and descriptive names appear in the sidebar that follows. We suspect you will recognize many of the anti-pattern names as development practices you were told to avoid when you learned to program.